

(c) Cadence Design Systems Inc. Do not distribute.

Module 10: Timing Analysis and Debug

(c) Cadence Design Systems Inc. Do not distribute.

Lab 10-1 Using Global Timing Debug Interface to Debug Timing Results

Objective: To run the timing analysis and fix a couple of timing violations.

Tempus™ Timing Signoff Solution is a timing signoff tool used to verify that the design meets your timing goals. In this lab, you will first rerun the previous session of Place & Route and continue it with running Tempus timing analysis inside of Innovus™.

This lab uses the following software:

- ◆ INNOVUS221 (22.10-p001_1)
- ◆ SSV221 (22.10-p001_1)

You will then run Tempus in the Timing Signoff mode.

- ◆ You can start the Tempus tool using the command *tempus -stylus*.
- ◆ You can start the Tempus TSO software using the command *tempus -stylus -eco*.

The following files are from the physical design lab and will serve as inputs to Tempus:

- ◆ **counter_netlist.v** – Gate-level netlist output after synthesis.
- ◆ **counter_sdc.sdc** – Constraint file generated during synthesis.
- ◆ **counter.view** – The implementation view definitions listed in this MMMC file.
- ◆ **gsclib045_tech.lef**, **gsclib045_macro.lef** – LEF files used in physical design.
- ◆ **slow_vdd1v0_basicCells.lib** and **fast_vdd1v0_basicCells.lib** – Timing libraries.

There are a few additional files in the directory that are not listed above.

Running Timing Analysis and Debugging in the Innovus Session

Innovus is a cockpit for a lot of implementation and verification tools front-to-back.

In this section, you will be running a Tempus-style timing analysis inside of Innovus.

1. Let us rerun the previous session of Place & Route inside the *STA* directory instead of using a saved session:

```
LINUX# cd STA
```

```
LINUX# innovus -stylus -files runPnR.tcl
```

At the end of this session, the design is routed and timing analyzed.

A screenshot of runPnR.tcl is shown here.

```
runPnR.tcl (~/.RTLtoGDSII/RTLtoGDSII_5.0/counter_design_database_45nm/STA) - GVIM17
File Edit Tools Syntax Buffers Window Help
runPnR.tcl counter.scandef
#@(#)CDS: CPE v20.10-p006
#@(#)CDS: IQuantus/TQuantus 19.1.3-s155 (64bit) Sun Nov 3 18:26:52 PST 2019 (Linux 2.6.32-431.11.2.el6.x86_64)

set db init_netlist_files ../physical_design/counter_netlist.v
set db init_lef_files {../lef/gscLib045_tech.lef ../lef/gscLib045_macro.lef}
set db init_power_nets VDD
set db init_ground_nets VSS
set db init_mmc_files counter.view
read_mmc counter.view
read_physical -lef {../lef/gscLib045_tech.lef ../lef/gscLib045_macro.lef}
read_netlist ../physical_design/counter_netlist.v -top counter
init_design

connect_global_net VDD -type pg_pin -pin_base_name VDD -inst_base_name *
connect_global_net VSS -type pg_pin -pin_base_name VSS -inst_base_name *

#Create Floorplan
create_floorplan -core_margins_by die -site CoreSite -core_density_size 1 0.4 2.5 2.5 2.5 2.5

#Pin Assignment
read_io_file pins.io

#Power Planning
set_db add_rings_skip_shared_inner_ring none ; set_db add_rings_avoid_short 1 ; set_db add_rings_ignore_rows 0 ; set_db add_rings_extend_over_row 0

#Add Rings
add_rings -type core_rings -jog_distance 0.6 -threshold 0.6 -nets {VDD VSS} -follow core -layer {bottom Metal11 top Metal11 right Metal10 left Metal10} -width 0.7 -spacing
.4 -offset 0.6

#Add Stripes
add_stripes -block ring_top_layer_limit Metal11 -max_same_layer_jog_length 0.44 -pad_core_ring_bottom_layer_limit Metal9 -set_to_set_distance 5 -pad_core_ring_top_layer_lim
it Metal11 -spacing 0.4 -merge_stripes_value 0.6 -layer Metal10 -block_ring_bottom_layer_limit Metal9 -width 0.3 -nets {VDD VSS}

# Create Power Rails with Special Route
route_special -connect core_pin -layer change_range { Metal1(1) Metal11(11) } -block pin target nearest target -core pin target first after_row_end -allow_jogging 1 -crosso
ver_via_layer_range { Metal1(1) Metal11(11) } -nets { VDD VSS } -allow_layer_change 1 -target_via_layer_range { Metal1(1) Metal11(11) }
# Read the Scan DEF
read_def counter.scandef
set_db reorder_scan_comp_logic true
-- INSERT --
38,11 37%
```

- Set the timing analysis mode by running the following commands:

```
set_db timing_analysis_type ocv
```

- Run setup-and-hold timing analysis by running the following commands:

```
time_design -post_route
```

Are there any setup violations?

Answer: _____

As you can see in the following timing summary, the design doesn't have any violating paths in setup mode.

time_design Summary				
Setup views included:				
WC				
Setup mode	all	reg2reg	default	
WNS (ns):	8.395	8.395	8.726	
TNS (ns):	0.000	0.000	0.000	
Violating Paths:	0	0	0	
All Paths:	40	15	25	

Let's run the hold check using the command:

```
time_design -post_route -hold
```

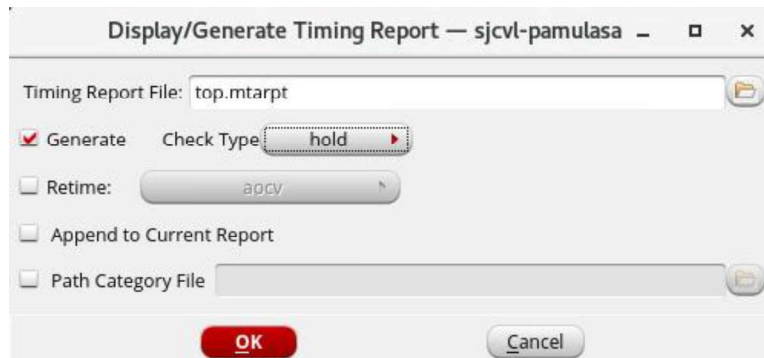
time_design Summary				
Hold views included: bc				
Hold mode	all	reg2reg	default	
WNS (ns):	-0.092	-0.025	-0.092	
TNS (ns):	-0.983	-0.158	-0.824	
Violating Paths:	17	8	9	
All Paths:	24	15	9	

Note: There are a lot of **hold violations** in the design. Let us see how to fix all of them in Tempus.

- If the graphical interface is not open, start it using the following command:

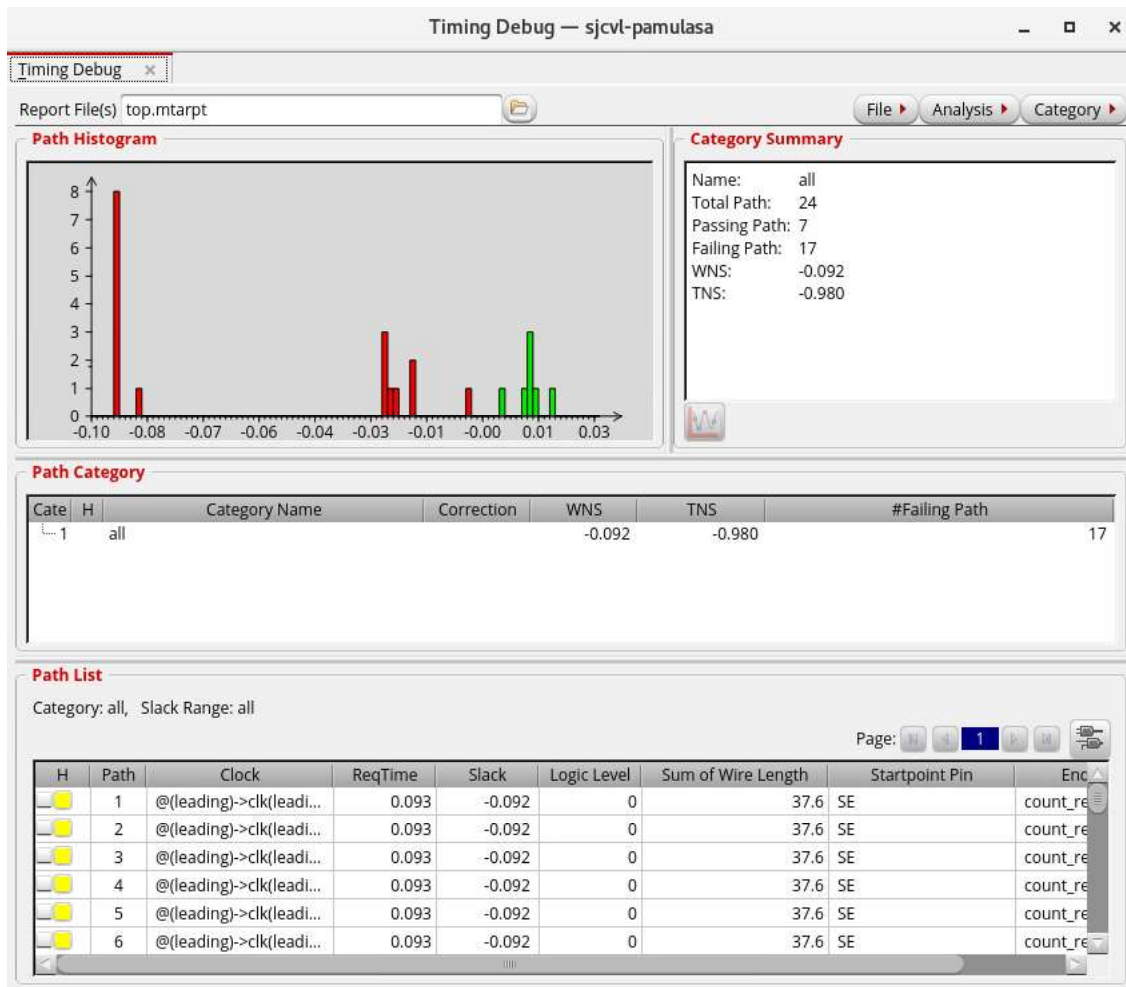
```
gui_show
```

- Run timing analysis using the **Timing – Debug Timing** menu.



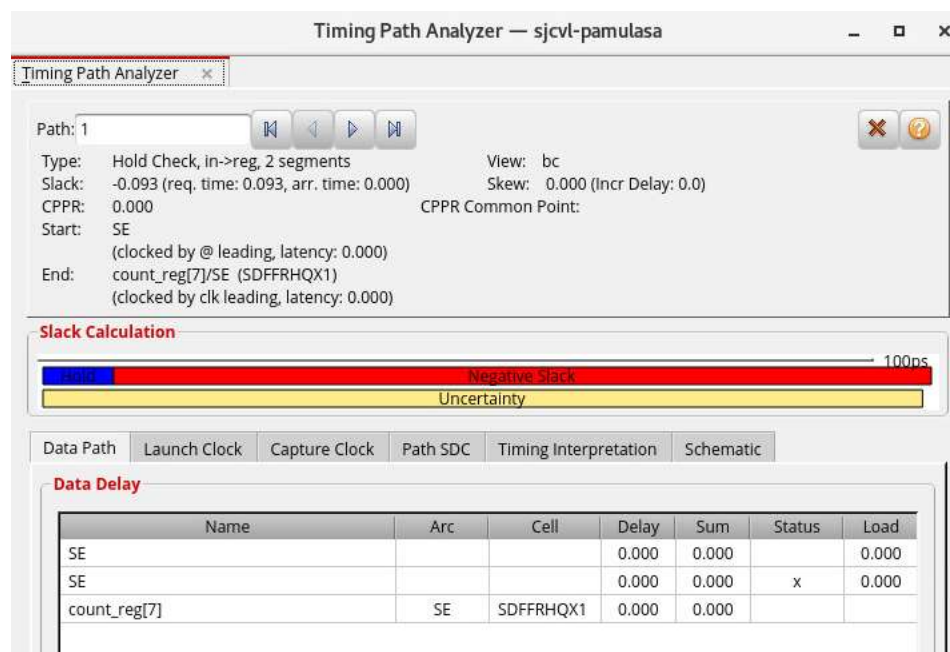
- Select **hold** in the Check Type field.
- Click **OK**.

The Timing Debug window opens.



c. **Right-click on path #1 in the Path List and select Show Timing Path Analyzer.**

You will see the Timing Path Analyzer window.



Note: From the Timing Path Analyzer, if you want to debug violating paths, you can **right-click** on any of the signals, and you will get additional options like interactive ECO, etc.

6. Save the design including the SPEF, DEF, and the libraries, by entering:

```
write_db -rc_extract -def postRoute
```

7. Keep this session open if you would like for debugging purposes. You can close it later.

Running an Independent Timing Analysis in Tempus

1. Start Tempus independently:

```
tempus -stylus
```

2. Load the Innovus database into Tempus using the following command:

```
read_db -physical_data postRoute
```

This loads the entire design along with the physical layout.

Open the **Layout** tab (click the  sign to see other available tabs) to confirm that the same layout from Innovus is also shown here in Tempus.

Once the design is loaded successfully, then generate the following reports.

3. Check the analysis coverage using the following command:

```
report_analysis_coverage
```

4. View the list of all constraint violations using the following command:

```
report_constraint -all_violators
```

5. Report the worst slack time for setup and hold, respectively, using the following commands:

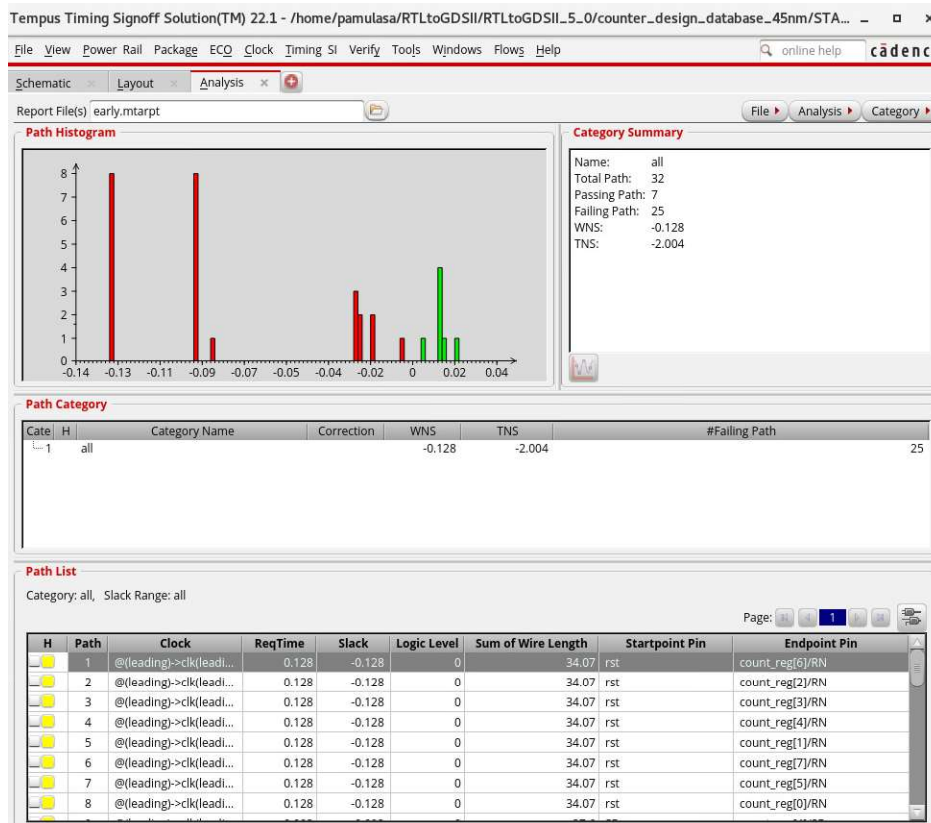
```
report_timing -late  
report_timing -early
```

6. Generate timing histograms using the following commands:

```
report_timing -early -max_paths 100 -output_format gtd > early.mtarpt  
read_timing_debug_report early.mtarpt
```

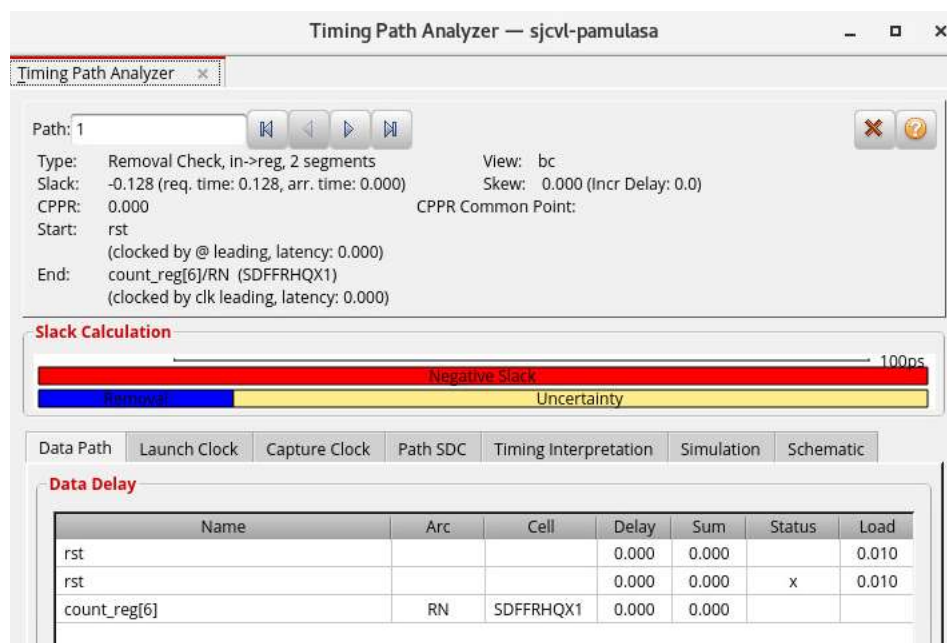
7. Open the **Analysis** tab (click the  sign to see other available tabs).

After the timing analysis is done, you can see the histogram for the hold analysis.

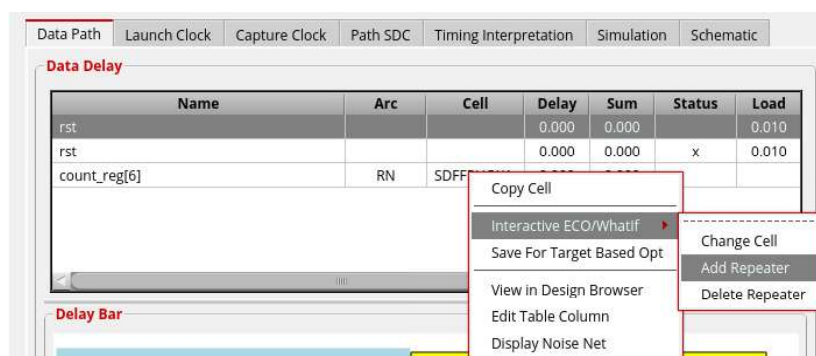


- a. Browse through the Path List with Startpoint Pin as **rst**, **right-click** on the path, and select **Show Timing Path Analyzer**.

You will see the Timing Path Analyzer window.

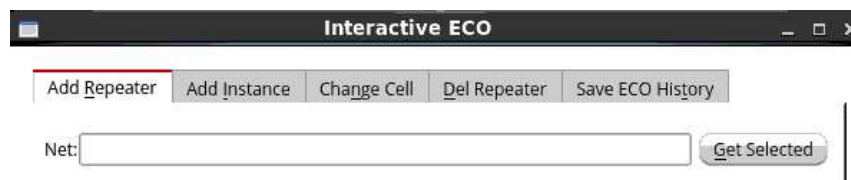


8. In the Timing Path Analyzer window, select the **rst** pin and **right-click** and select **Interactive ECO/WhatIf – Add Repeater**.



The Interactive ECO window opens, which will allow you to **add repeaters**.

9. In the Interactive ECO window:
 - a. Click **Get Selected** to populate the net information, or just enter **rst**.



- b. In the New Cell field, select **BUFX2**.



- c. Choose **early** from the *Evaluation Type*.
- d. Click **Eval All** to choose which buffer is suitable for the path that adds more delay and reduces the hold slack.

Notice the list of buffer suggestions appears at the Tempus terminal.

```
@tempus 7> eco add repeater -cells BUX2 -net rst -evaluate.all -sort by slack -early
#####
# Design Name: counter
# Design Mode: 65nm
# Analysis Mode: MMMC 0CV
# Parasitics Mode: SPEF/RCDB
# Signoff Settings: SI On
#####
```

Library-cell	Area	Timing-slack	Worst-Slack-View	Worst-Slack-Dir
Reference	0.0000	-0.1281	bc	rise
CLKBUX6(buf)	3.0780	-0.1049	bc	rise
BUFX6(buf)	3.0780	-0.1048	bc	rise
BUFX12(buf)	5.1300	-0.1036	bc	rise
CLKBUX12(buf)	5.1300	-0.1036	bc	rise
BUFX8(buf)	3.7620	-0.1033	bc	rise
CLKBUX8(buf)	3.7620	-0.1033	bc	rise
BUFX16(buf)	6.8400	-0.1032	bc	rise
CLKBUX16(buf)	6.8400	-0.1032	bc	rise
CLKBUX20(buf)	8.2080	-0.1022	bc	rise
BUFX20(buf)	8.2080	-0.1021	bc	rise
BUFX3(buf)	2.0520	-0.1007	bc	rise
CLKBUX3(buf)	2.0520	-0.1006	bc	rise
CLKBUX4(buf)	2.3940	-0.1004	bc	rise
BUFX4(buf)	2.3940	-0.1003	bc	rise
CLKBUX2(buf)	1.7100	-0.0957	bc	rise
BUFX2(buf)	1.7100	-0.0956	bc	rise

From the evaluation list, we can observe that Slack can be reduced to a maximum extent to **-0.0956** with the cell **BUFX2**.

Note: Add only BUFX* cells to fix the timing violation, do not add any CLKBUFX* cells in the design.

- e. Now, select the **BUFX2** cell in the Interactive ECO.
- f. Click **Eval Only**.
- g. View the messages by the **<tempus>** prompt.

Note: Ensure that there is a change in the Slack (it is negative). The result should be an improvement in Slack **-0.0956** when you Apply the repeater addition.

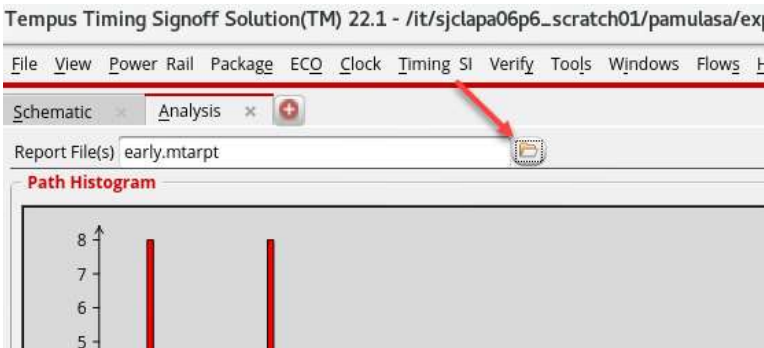
Library-cell rans-Sink	Area	Timing-slack	Worst-Slack-View	Worst-Slack-Dir	Trans	Trans-slack
Reference eg[7]/RN	0.0000	-0.1281	bc	rise	0.0040	0.2760
BUFX2(buf) eg[7]/RN	1.7100	-0.0956	bc	rise	0.1168	0.1632

10. Click **Apply**.

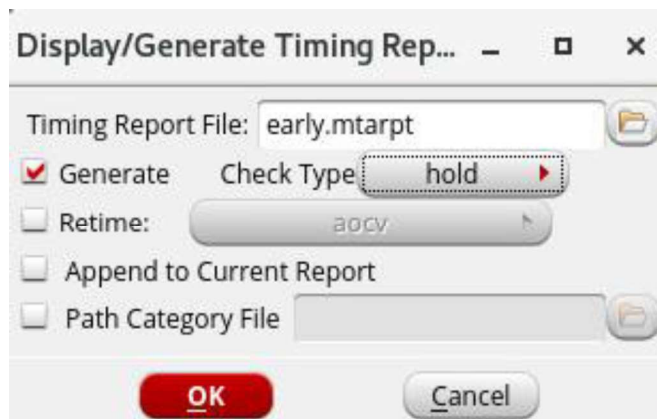
The slack improvement is clearly observable. Let us see now analyze the results using the Timing Path Analyzer window.

Note: Ignore the *ERROR: (IMPSP-365)* if any.

11. Rerun the timing report using the below icon from the **Analysis** window and compare the WNS.
- a. Click the load icon from the *Report Files(s)* field



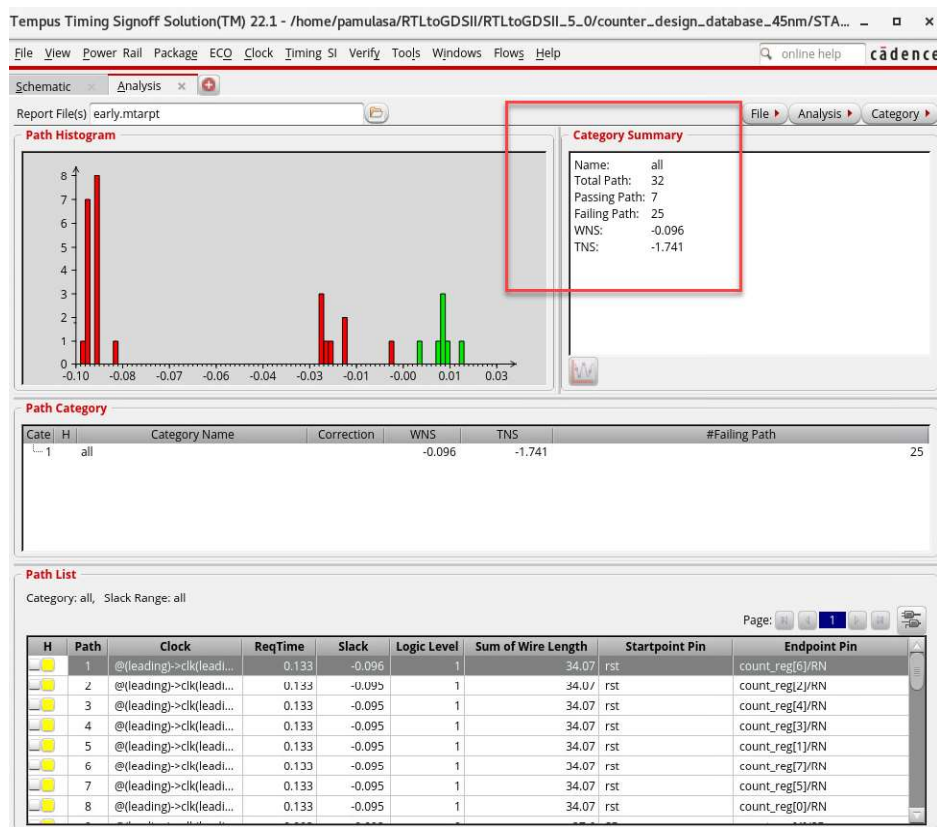
The Display/Generate Timing Report window appears.



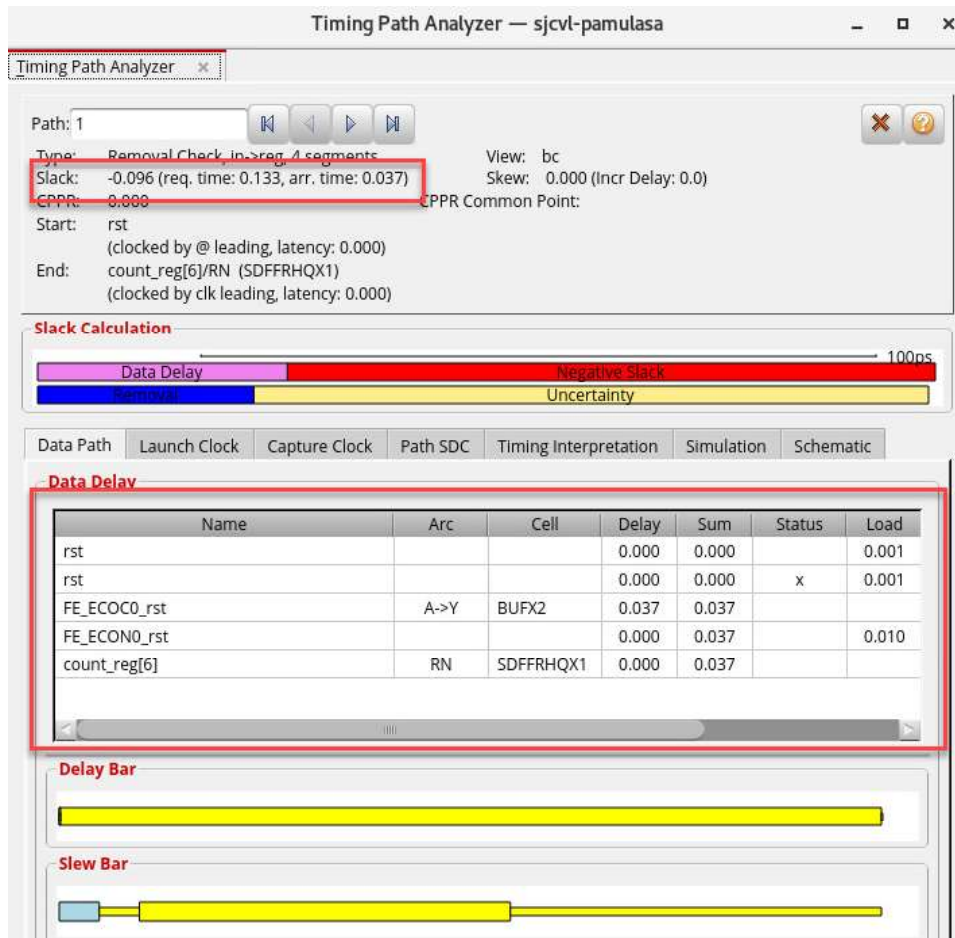
- b. Choose the Timing Report File *early.mtarpt*.
- c. Select **hold** from Check Type.
- d. Click **OK**.

12. Notice the improvement in timing using the below screenshots.

Notice the improvement in the **WNS** in the **Analysis** window.

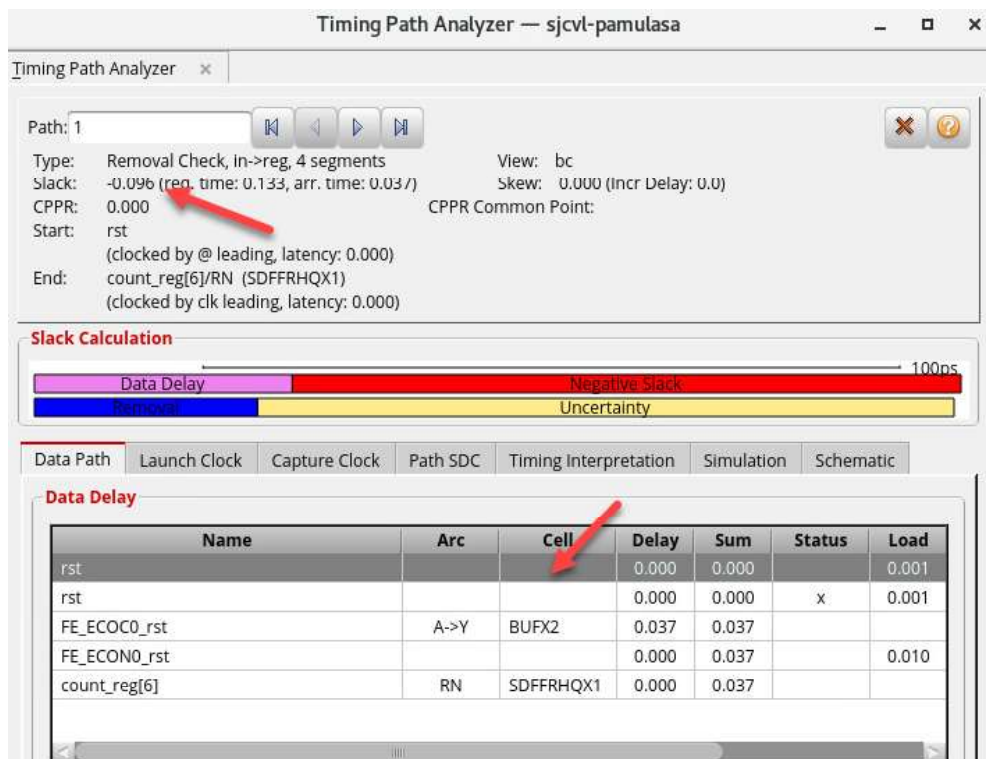
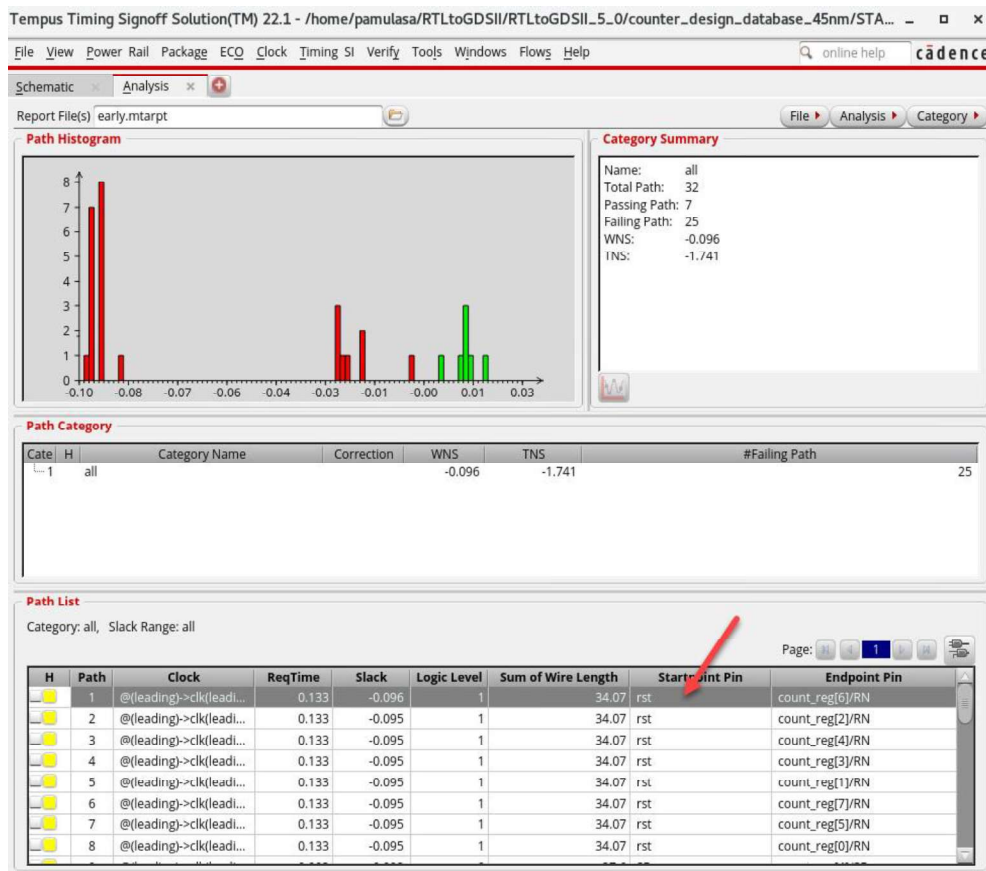


Notice the improvement in Hold Slack for the **rst** pin in the **Timinig Path Analyzer** window.



13. As we can see, the Slack is still negative (**-0.096**), and we need to add a buffer on the same net of the pin **rst**, until we get the Zero/Positive Slack for the pin **rst**.

14. So, In the **Analysis** window, select the **rst** pin and **right-click** again on the path, and select **Show Timing Path Analyzer**.



15. Repeat Steps 8, 9, and 10 again by selecting the first path of the **rst** pin and choosing **Add Repeater** from the Timing Path Analyzer window.

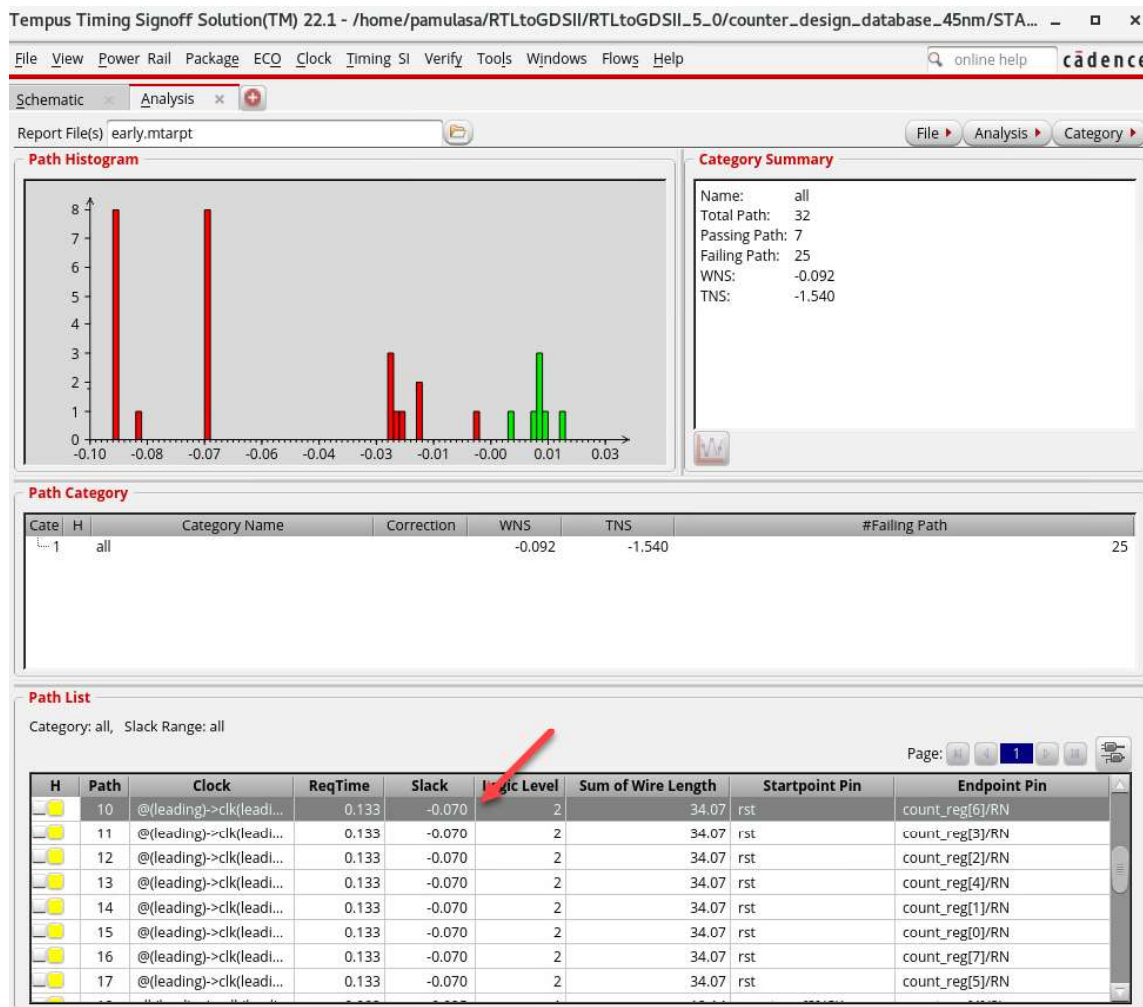
Click **Eval all** and choose the buffer cell (BUFX*).

```
@tempus 20> eco add_repeater -cells BUFX20 -net rst -evaluate_all -sort_by slack -early
#####
# Design Name: counter
# Design Mode: 65nm
# Analysis Mode: MMMC OCV
# Parasitics Mode: SPEF/RCDB
# Signoff Settings: SI On
#####
-----
Library-cell          Area      Timing-slack  Worst-Slack-View  Worst-Slack
-----
Reference             0.0000     -0.0956      bc                 rise
BUFX3(buf)            2.0520     -0.0767      bc                 rise
CLKBUFX3(buf)         2.0520     -0.0767      bc                 rise
CLKBUFX6(buf)         3.0780     -0.0767      bc                 rise
BUFX6(buf)            3.0780     -0.0766      bc                 rise
CLKBUFX2(buf)         1.7100     -0.0759      bc                 rise
BUFX2(buf)            1.7100     -0.0757      bc                 rise
CLKBUFX4(buf)         2.3940     -0.0743      bc                 rise
BUFX4(buf)            2.3940     -0.0742      bc                 rise
BUFX8(buf)            3.7620     -0.0739      bc                 rise
CLKBUFX8(buf)         3.7620     -0.0739      bc                 rise
BUFX12(buf)           5.1300     -0.0731      bc                 rise
CLKBUFX12(buf)        5.1300     -0.0731      bc                 rise
BUFX16(buf)           6.8400     -0.0721      bc                 rise
CLKBUFX16(buf)        6.8400     -0.0721      bc                 rise
BUFX20(buf)           8.2080     -0.0705      bc                 rise
CLKBUFX20(buf)        8.2080     -0.0705      bc                 rise
-----
```

Observe the **BUFX20** cell is the best delay cell to add to the path to reduce the Hold Timing Slack to **-0.0705**.

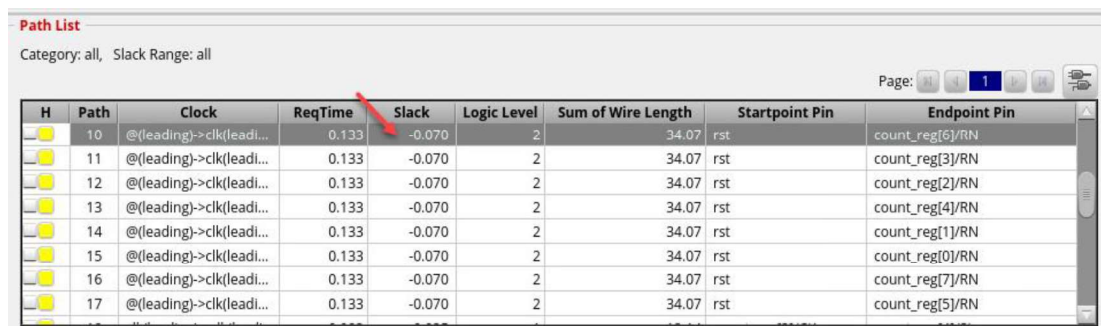
Note: If you have an area constraint or less area in your design, then choose a cell that adds moderate delay and less area according to the design requirement.

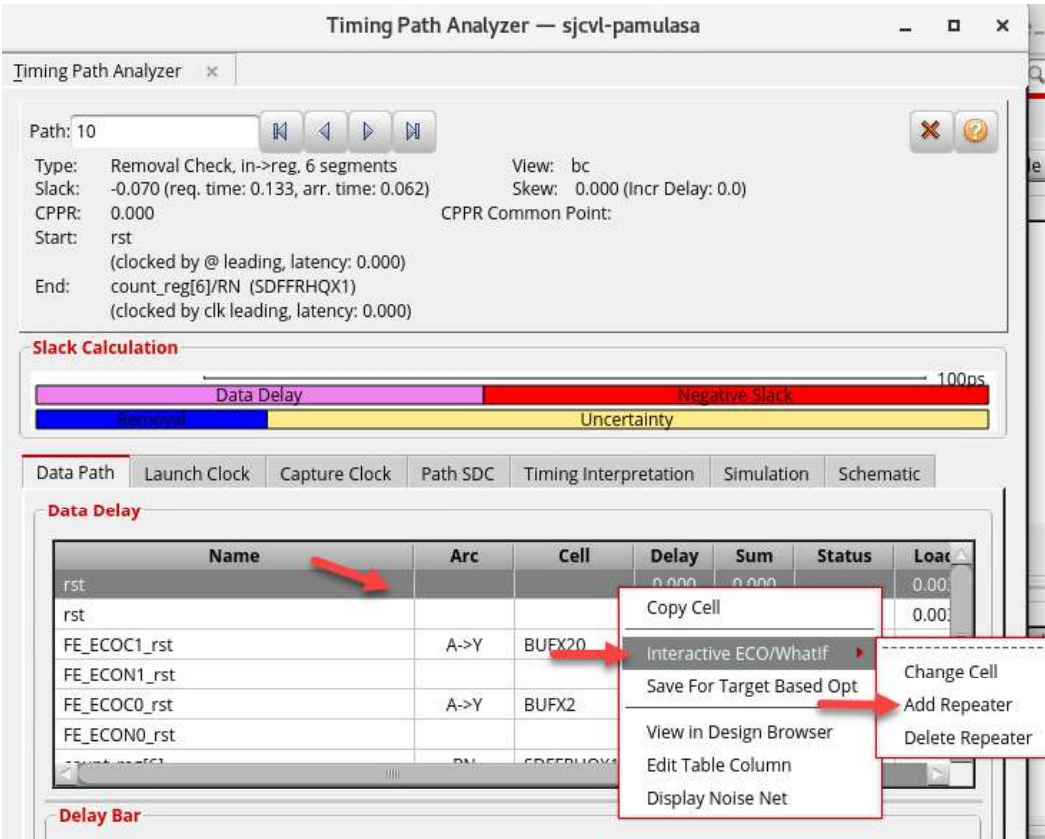
16. After adding the **BUFX20** cell, rerun the **Timing Debug** report as shown in **Step 11**.



Note: As we can see, the Slack is still negative (**-0.070**), and we need to add buffers on the same net of the pin **rst**, until we get the Zero/Positive Slack for the pin **rst**.

17. Repeat Steps 7, 8, 9 and 10 again by selecting the first path of the **rst** pin from the **Analysis** window and choosing **Add Repeater** from the Timing Path Analyzer window.





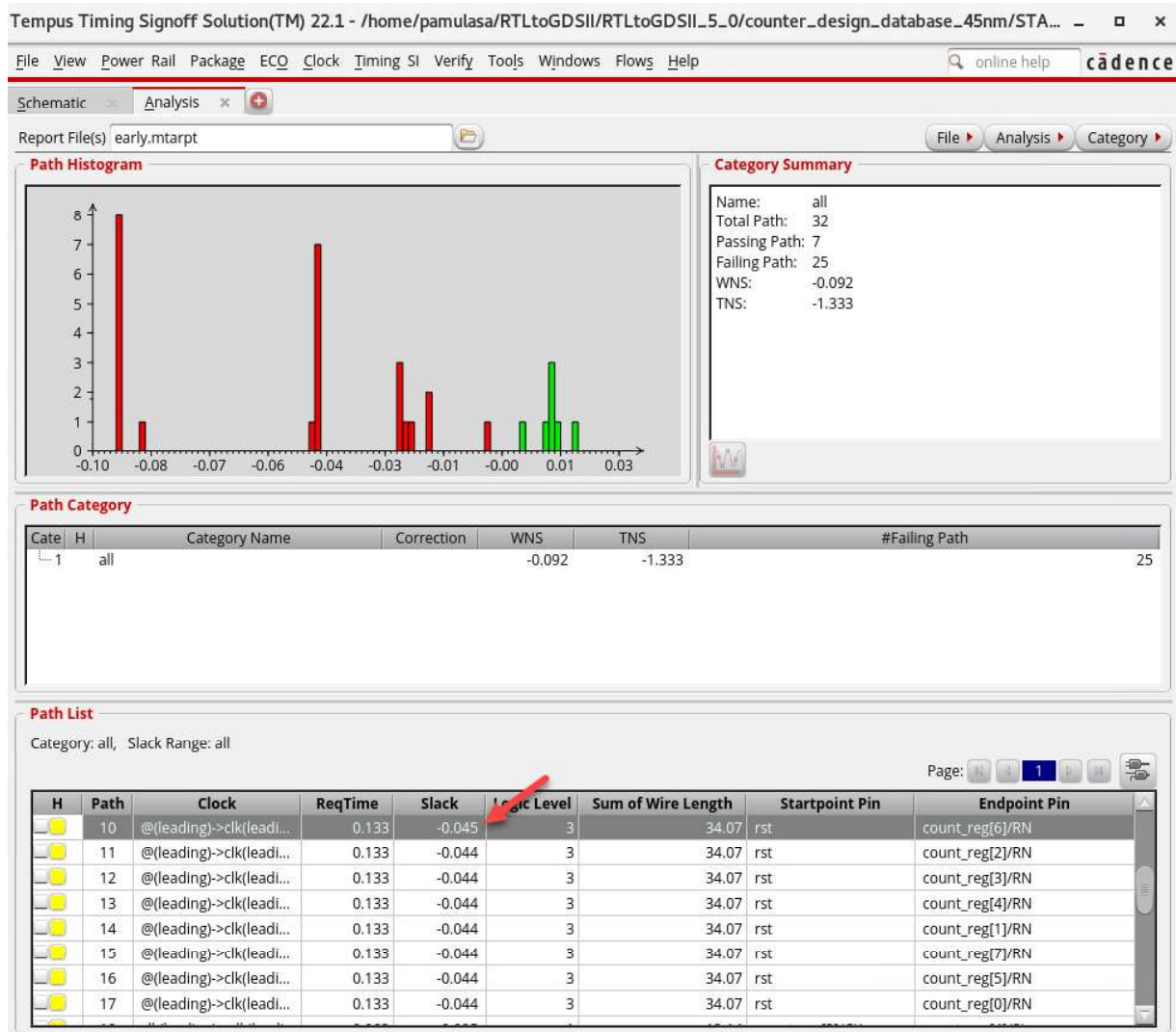
Click **Eval all** and choose the buffer cell (BUFX*).

Library-cell			
	Area	Timing-slack	Worst-Slack-View
Reference	0.0000	-0.0704	bc
CLKBUFX6(buf)	3.0780	-0.0499	bc
BUFX6(buf)	3.0780	-0.0498	bc
BUFX3(buf)	2.0520	-0.0479	bc
CLKBUFX3(buf)	2.0520	-0.0479	bc
BUFX8(buf)	3.7620	-0.0473	bc
CLKBUFX8(buf)	3.7620	-0.0473	bc
BUFX12(buf)	5.1300	-0.0469	bc
CLKBUFX12(buf)	5.1300	-0.0469	bc
CLKBUFX4(buf)	2.3940	-0.0464	bc
BUFX4(buf)	2.3940	-0.0462	bc
BUFX16(buf)	6.8400	-0.0459	bc
CLKBUFX16(buf)	6.8400	-0.0459	bc
CLKBUFX2(buf)	1.7100	-0.0450	bc
BUFX2(buf)	1.7100	-0.0448	bc
BUFX20(buf)	8.2080	-0.0444	bc
CLKBUFX20(buf)	8.2080	-0.0444	bc

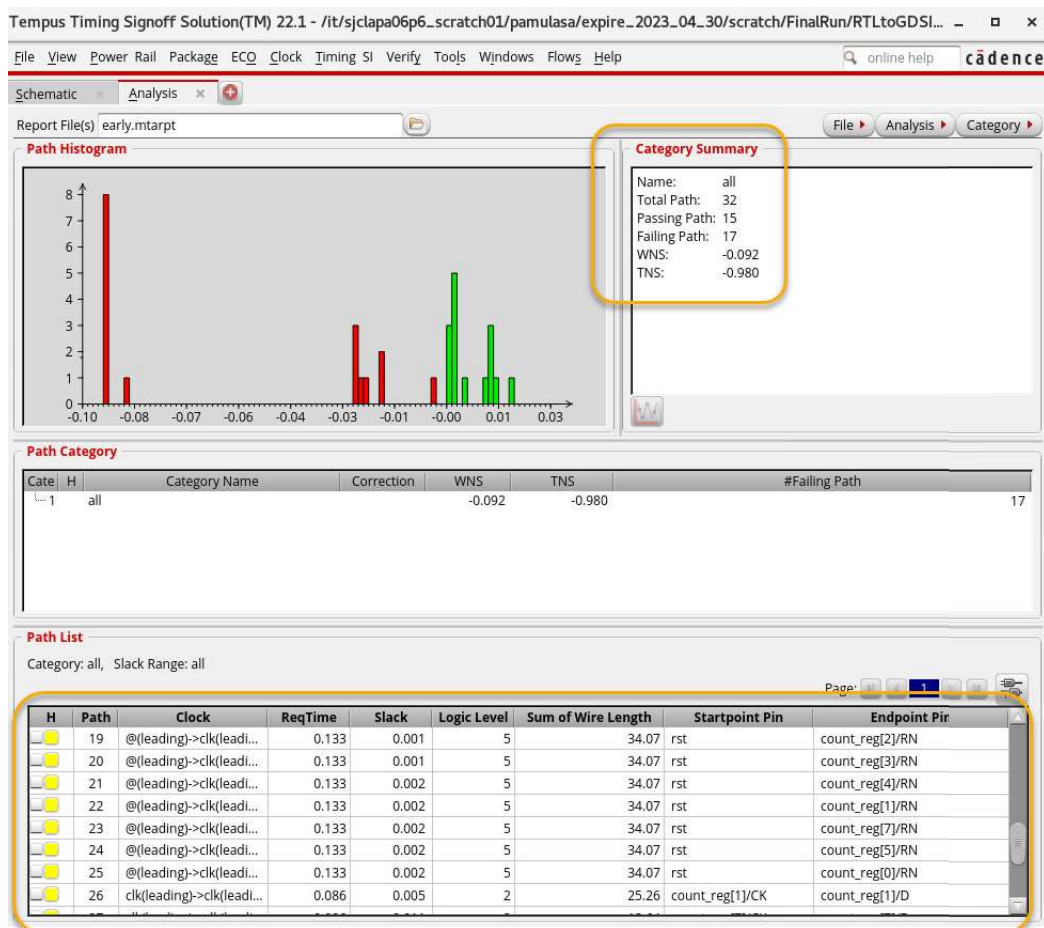
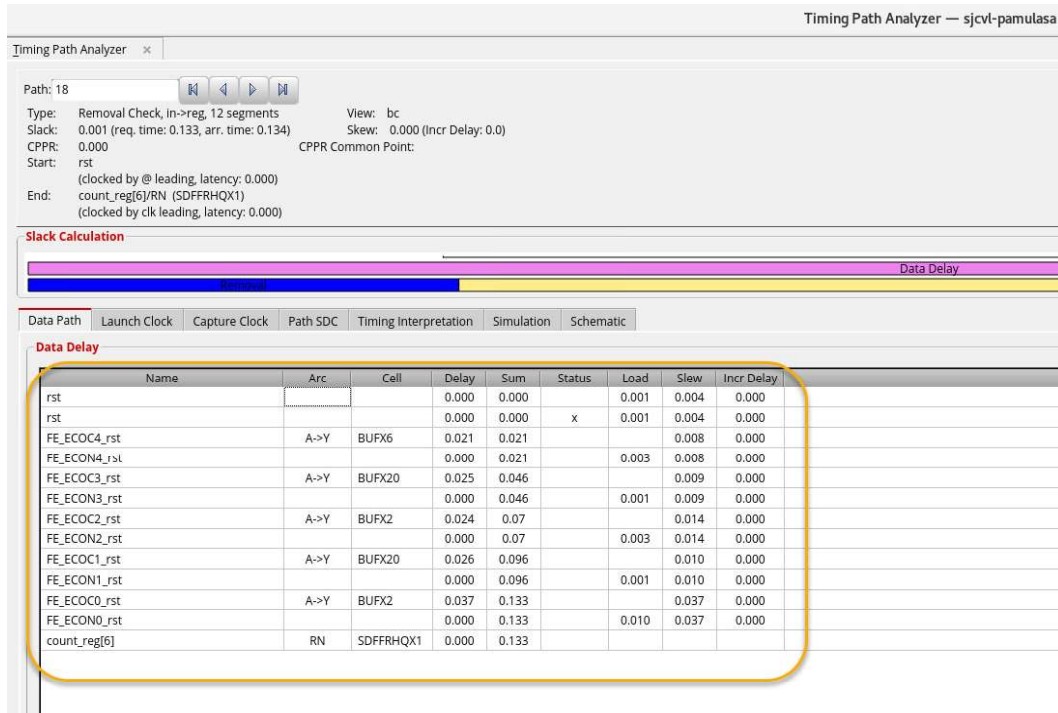
Note: Observe that **BUFX20** cell is the best delay cell to add on the path to reduce the timing slack but has the highest area penalty. So, ensure that if you have the area constraint or less area is present in your design, then go for the cell, which adds moderate delay with a lesser area overhead.

So, choose the **BUFX2** cell in this scenario because the area of **BUFX2** is less than the **BUFX20** cell, and also Timing-Slack reduction is not much different between those two cells.

18. After adding the **BUFX2** cell, rerun the **Timing Debug** report and analyze the Hold Slack as shown in **Step 11**



19. Likewise, add more buffers on the same path, analyze the Hold slack for the pin **rst**, and notice the reduction in **WNS**.



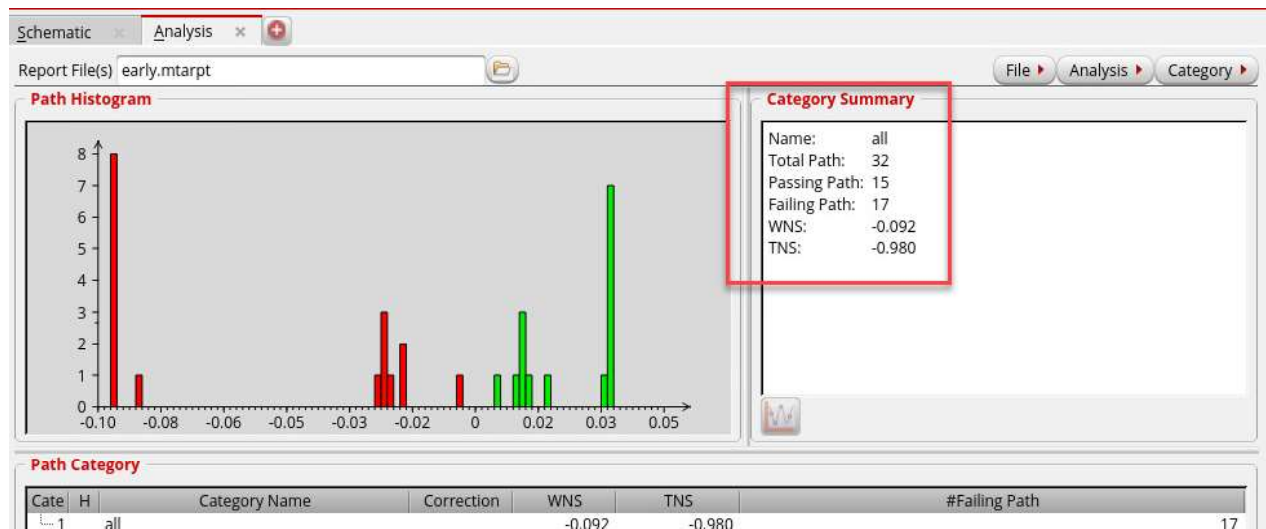
20. Now, check the analysis coverage using the following command to ensure the **Setup Slack** is not affected by the changes made in the Hold Violated Paths.

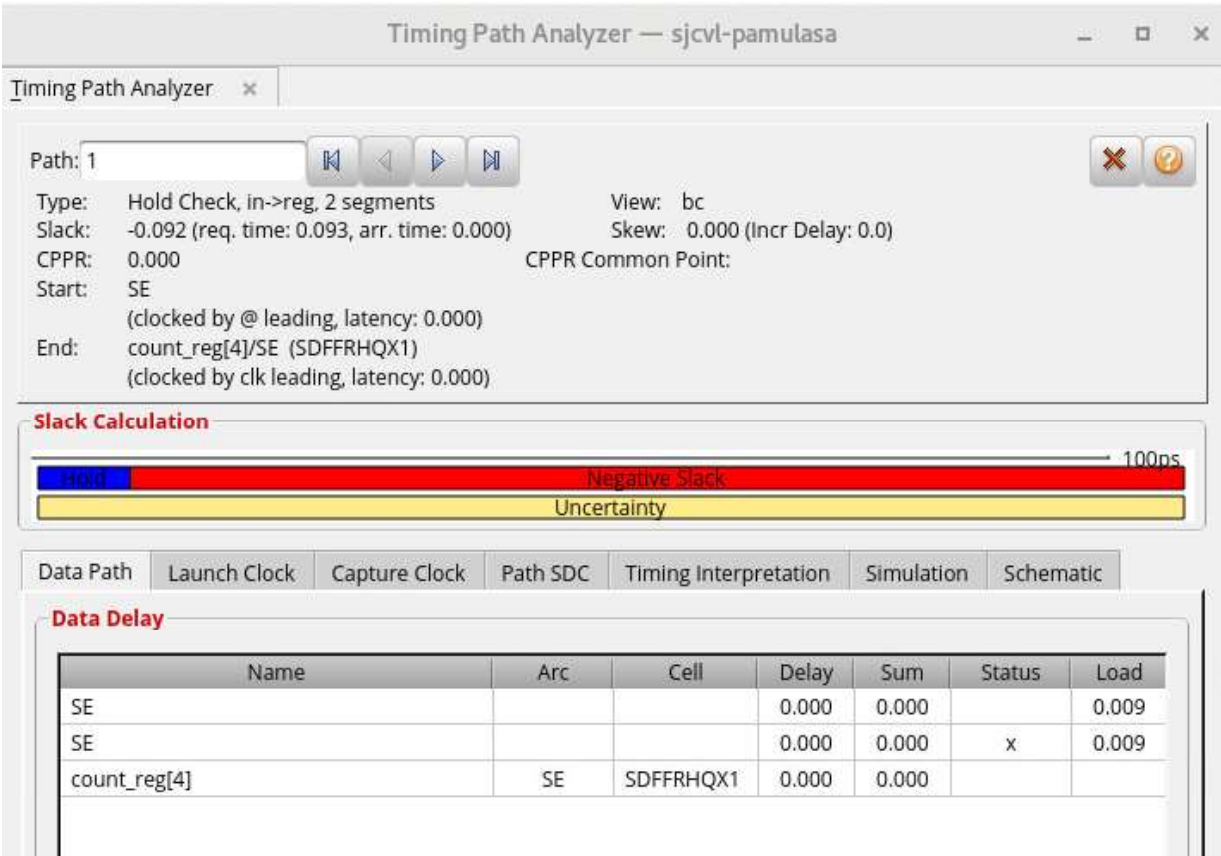
```
report_analysis_coverage

@tempus 51> report_analysis_coverage
#####
# Generated by:      Cadence Tempus 22.10-p001 1
# OS:               Linux x86_64(Host ID sjcvl-pamulasa)
# Generated on:      Mon Apr 24 03:55:29 2023
# Design:           counter
# Command:          report_analysis_coverage
#####
-----
                        TIMING CHECK COVERAGE SUMMARY
-----
Check Type              No. of  Met          Violated      Untested
                        Checks
-----
ExternalDelay (Late)    8       8 (100%)      0 (0%)        0 (0%)
Hold                   24       7 (29%)      17 (70%)      0 (0%)
PulseWidth            24      16 (66%)      0 (0%)        8 (33%)
Recovery               8       8 (100%)      0 (0%)        0 (0%)
Removal               8       8 (100%)      0 (0%)        0 (0%)
Setup                  24      24 (100%)      0 (0%)        0 (0%)
-----
```

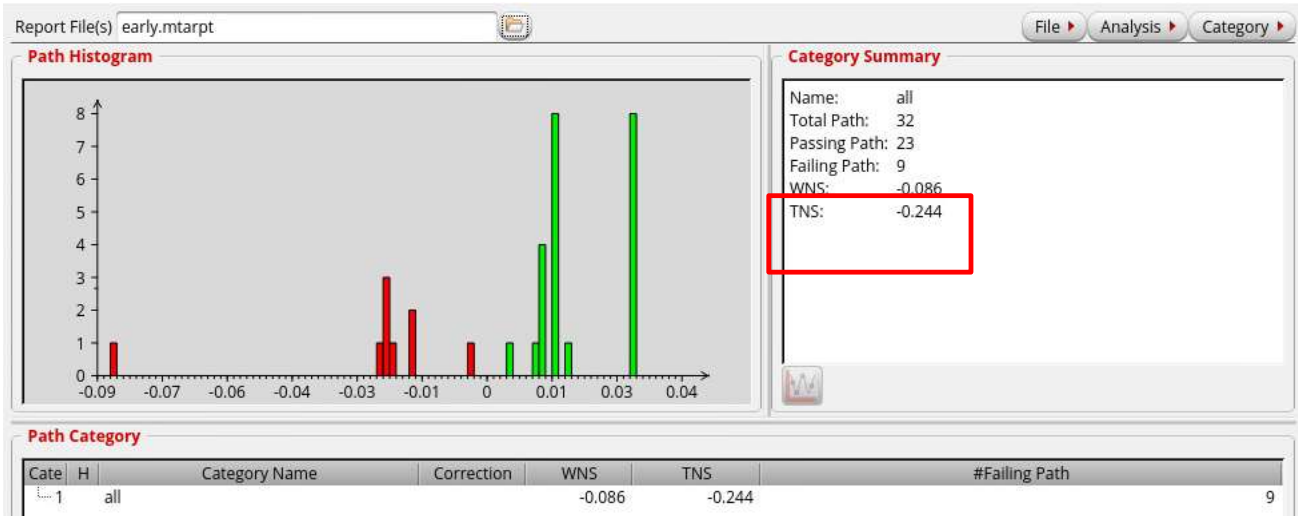
21. Now. Try a similar exercise by buffering the **SE** pin to fix the **Hold Slack** Violations and compare the results.

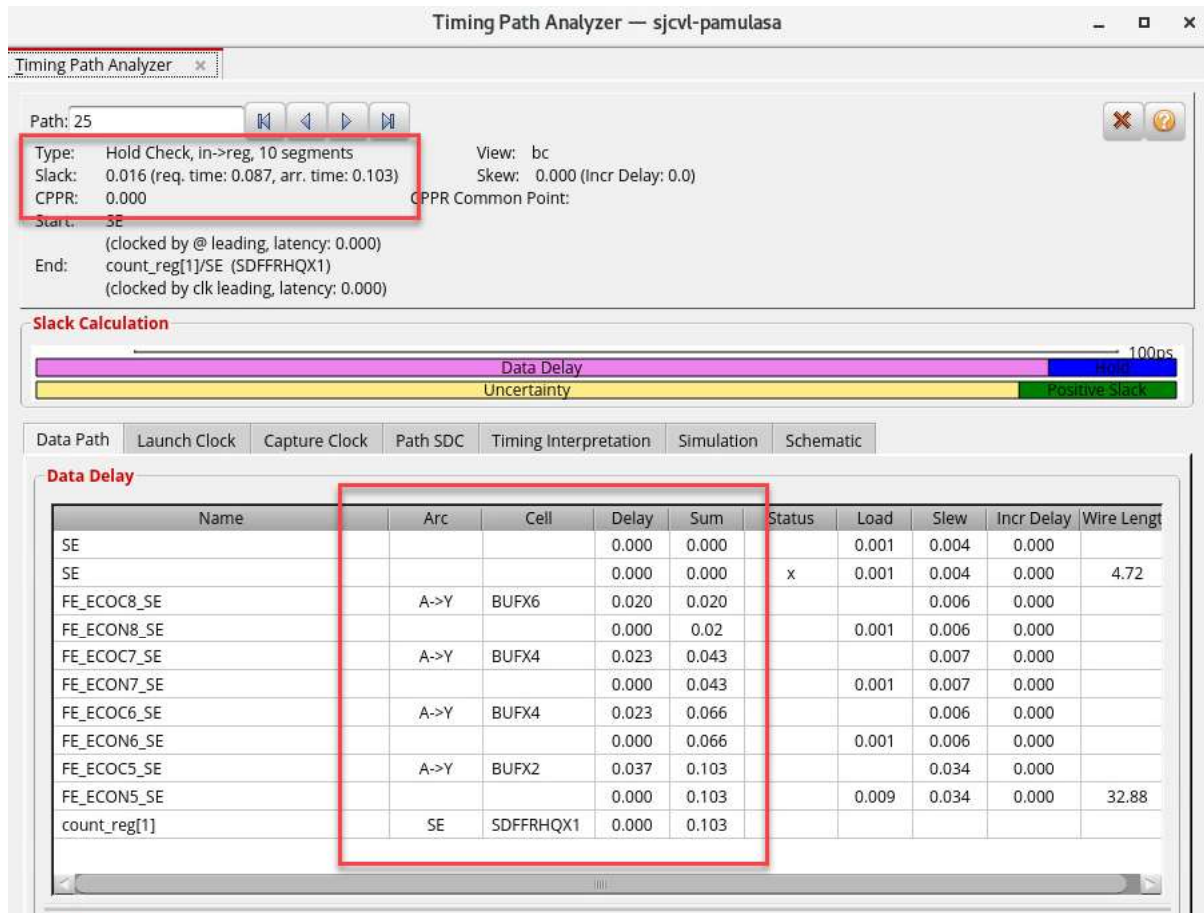
Earlier:



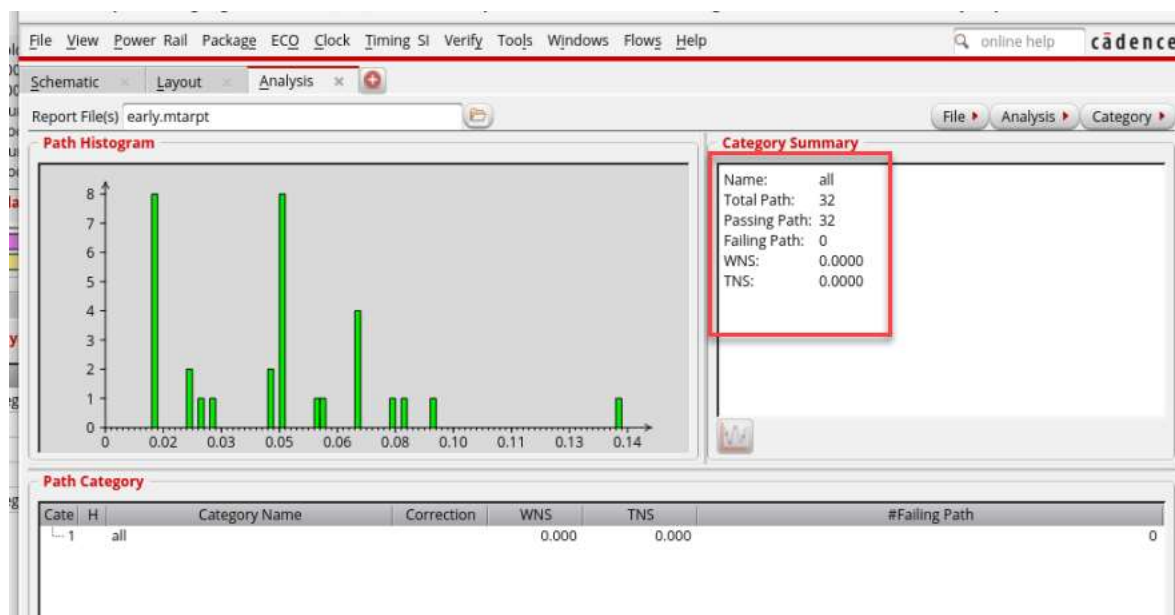


Later:





22. Similarly, fix the remaining timing violations. As a result, it will show zero failing paths in the final timing report, as shown in the figure below.



Summary

1. You can run timing analysis from within Innovus. You can also evaluate and create timing-fixing ECOs interactively from within Tempus.
2. You can run independent timing analysis from within Tempus. Independent analysis frees up Innovus to do other things.

